

0.1. Tổng quan về ngôn ngữ lập trình JavaScript

0.1.1. Khái niệm và đặc điểm của JavaScript

JavaScript là ngôn ngữ lập trình thường được sử dụng khi phát triển website. Ngôn ngữ này ra đời vào những năm 1990 nhằm tăng khả năng tương tác cho các trang web và dần trở thành công nghệ quan trọng trong lập trình hiện đại.

Hiện nay, JavaScript được ứng dụng ở cả Frontend và Backend nhờ khả năng hoạt động linh hoạt trên nhiều nền tảng khác nhau.

Đặc điểm của JavaScript:

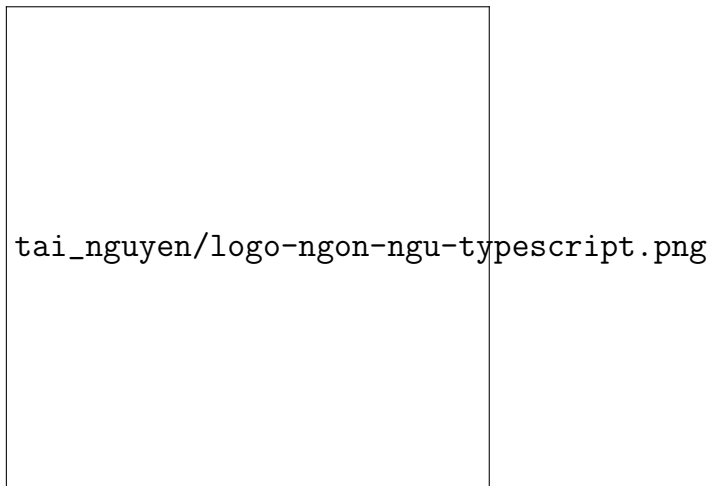
- **Cú pháp tối giản, thân thiện và dễ tiếp cận:** JavaScript sở hữu cấu trúc mã nguồn tường minh, chịu ảnh hưởng lớn từ ngôn ngữ C nên rất trực quan đối với người mới bắt đầu.
- **Đa mô hình lập trình, tối ưu hóa hướng đối tượng và xử lý bất đồng bộ:** JavaScript là một ngôn ngữ đa mô hình, hỗ trợ mạnh mẽ lập trình hướng đối tượng (OOP) dựa trên cơ chế Prototype, kết hợp linh hoạt với lập trình hàm (Functional Programming).
- **Nền tảng kiến tạo giao diện web tương tác động cao:** Bằng cách can thiệp và thao tác trực tiếp vào mô hình tài liệu DOM (Document Object Model), JavaScript cho phép thay đổi nội dung, cấu trúc và định dạng trang web theo thời gian thực dựa trên hành vi của người dùng.

Vai trò của Node.js:

- **Nền tảng tối ưu cho ứng dụng Server-side hiệu năng cao, thời gian thực:** Node.js áp dụng mô hình Kiến trúc hướng sự kiện (Event-driven) và Cơ chế I/O không chặn (Non-blocking I/O), xử lý hàng ngàn kết nối đồng thời với mức tiêu hao bộ nhớ cực thấp. Đặc điểm này biến Node.js thành lựa chọn tối ưu để xây dựng các hệ thống API (RESTful, GraphQL) và các ứng dụng trao đổi dữ liệu thời gian thực như IoT.
- **Chuẩn hóa ngôn ngữ toàn diện (Full-stack JavaScript):** Node.js xóa bỏ rào cản giữa Frontend và Backend, kiến tạo xu hướng Full-stack JavaScript.

0.1.2. Sự ra đời của TypeScript (TS)

TypeScript là ngôn ngữ lập trình mã nguồn mở do Microsoft phát triển. Về bản chất, nó được định nghĩa là một “superset” của JavaScript, cung cấp hệ thống định kiểu tĩnh (Static Typing) mà JavaScript thuần túy còn thiếu. TypeScript không thay thế JavaScript mà cung cấp thêm một lớp bảo vệ vững chắc, giúp quản lý mã nguồn thống nhất hơn.



Hình 1. Logo ngôn ngữ TypeScript

TypeScript hỗ trợ phát triển các dự án quy mô lớn thông qua:

- Cho phép kiểm tra lỗi trong quá trình biên dịch trước khi chương trình được thực thi.
- Tăng khả năng phối hợp giữa nhiều lập trình viên và giúp việc bảo trì hệ thống thuận tiện hơn.
- Mã nguồn TypeScript sẽ được chuyển thành mã JavaScript để trình duyệt có thể thực thi.
- Cung cấp khả năng sử dụng các tính năng mới của ECMAScript nhưng vẫn duy trì tính tương thích trên nhiều môi trường khác nhau.

0.1.3. Các đặc tính kỹ thuật nổi bật của TypeScript

TypeScript được xây dựng nhằm mở rộng khả năng của JavaScript thông qua việc bổ sung nhiều cơ chế hỗ trợ kiểm soát mã nguồn. Một số đặc điểm tiêu biểu gồm:

- **Hệ thống kiểu dữ liệu (Type System):** Cho phép khai báo kiểu rõ ràng cho biến, tham số và giá trị trả về của hàm. Các kiểu dữ liệu thường dùng gồm: string, number, boolean, array, enum, tuple và any.
- **Interfaces và Type Aliases:** Hỗ trợ mô tả cấu trúc dữ liệu và định nghĩa khuôn mẫu cho các đối tượng trong hệ thống.
- **Lập trình hướng đối tượng (OOP):** TypeScript hỗ trợ đầy đủ Class, Inheritance, Interface và Encapsulation thông qua các phạm vi truy cập như public, private và protected.

0.1.4. So sánh JavaScript và TypeScript trong phát triển dự án lớn

JavaScript có ưu điểm về tính linh hoạt và dễ tiếp cận. Tuy nhiên, khi sử dụng ở các hệ thống phức tạp, việc thiếu cơ chế kiểm soát kiểu dữ liệu rõ ràng có thể làm tăng nguy cơ phát sinh lỗi. TypeScript giải quyết vấn đề này qua ba điểm mấu chốt:

1. **Cơ chế phát hiện lỗi sớm:** TypeScript sở hữu trình biên dịch giúp rà soát toàn bộ

cú pháp và cấu trúc dữ liệu ngay trong quá trình phát triển (Compile-time), ngăn chặn lỗi trước khi triển khai.

2. **Nâng cao hiệu suất làm việc nhóm:** Hệ thống kiểu tường minh kích hoạt các tính năng như IntelliSense, Auto-complete, giúp lập trình viên cộng tác hiệu quả hơn.
3. **Kiến trúc mở rộng dài hạn:** TypeScript áp dụng Interface, Namespace và Generics, giúp module hóa hệ thống một cách khoa học, phù hợp cho các sản phẩm công nghệ dài hạn.

0.2. Thư viện ReactJS và Thuật toán Virtual DOM

0.2.1. Giới thiệu về thư viện ReactJS

ReactJS là một thư viện JavaScript được tạo ra bởi Facebook, tập trung vào lớp hiển thị (View) và mô hình thành phần (Component-based). Nó giúp xây dựng các ứng dụng web với dữ liệu thay đổi liên tục một cách hiệu quả và dễ bảo trì.

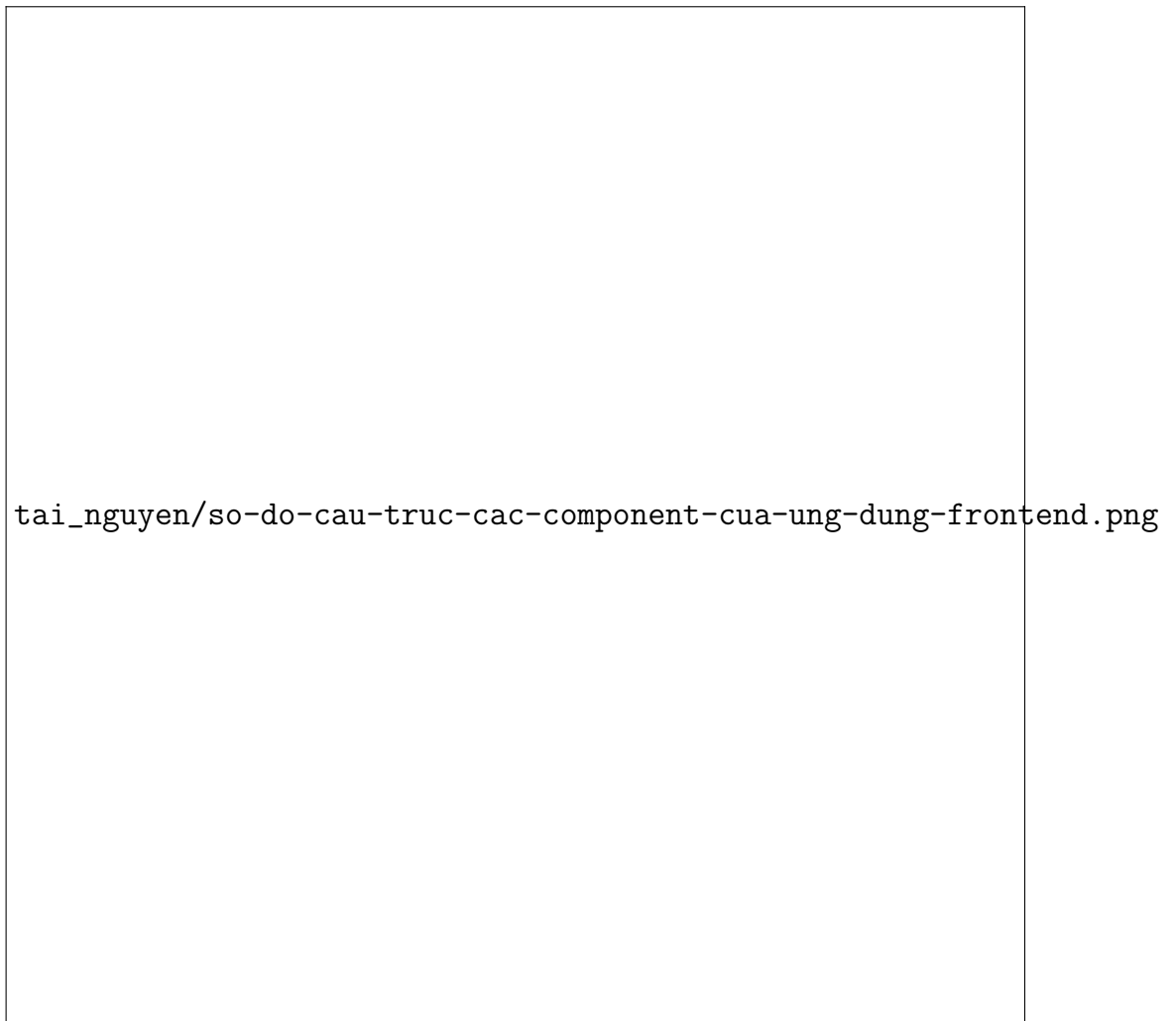
Các triết lý cốt lõi của ReactJS:

- **Declarative (Lập trình khai báo):** Lập trình viên chỉ cần mô tả trạng thái mong muốn của giao diện, React tự động quản lý việc cập nhật và render lại các thành phần tương ứng.
- **Cú pháp JSX (JavaScript XML):** Kết hợp sức mạnh của JavaScript với sự trực quan của cấu trúc HTML, giúp mã nguồn giao diện trở nên mạch lạc và dễ đọc hơn.
- **Luồng dữ liệu một chiều (Unidirectional Data Flow):** Dữ liệu luôn được truyền từ cha xuống con thông qua Props, giúp kiểm soát luồng dữ liệu minh bạch và hạn chế lỗi logic.

0.2.2. Kiến trúc thành phần của ReactJS (Component)

Kiến trúc của ReactJS xoay quanh khái niệm Component:

- **Tính đóng gói (Encapsulation):** Mỗi thành phần tự quản lý trạng thái riêng, giúp mã nguồn trở nên sạch sẽ và dễ quản lý.
- **Tính tái sử dụng (Reusability):** Các component có thể được đóng gói và sử dụng lại ở những vị trí khác nhau trong ứng dụng, giúp giảm thiểu trùng lặp mã nguồn.



Hình 2. Sơ đồ cấu trúc các Component của ứng dụng Frontend

0.2.3. Quản lý luồng dữ liệu (*State* và *Props*)

- **Props (Properties):** Là các thuộc tính được truyền từ thành phần cha xuống tới thành phần con. Props là dữ liệu chỉ đọc (Immutable), giúp cho luồng của hệ thống minh bạch.
- **State:** Là các biến lưu thông tin của Component, có khả năng thay đổi theo thời gian dựa trên thao tác của người dùng. Khi State thay đổi, React tự tiến hành cập nhật giao diện (Re-render).

0.3. Framework Next.js và Các phương thức Rendering tiên tiến

0.3.1. Giới thiệu về Framework Next.js

Next.js là một framework dựa trên React, cung cấp các tính năng cần thiết để xây dựng các ứng dụng web sẵn sàng cho sản xuất (Production-ready). Các tính năng cốt lõi bao gồm:

- **Hệ thống Routing (File-based Routing):** Tự động tạo ra các đường dẫn dựa trên cấu trúc thư mục và tệp tin, giúp quản lý cấu trúc trang web dễ dàng mở rộng.
- **Tối ưu hóa tài nguyên (Optimization):** Hỗ trợ tự động tối ưu hóa hình ảnh, phong

chữ và các đoạn mã script bên thứ ba.

- **Hỗ trợ TypeScript:** Tích hợp sẵn TypeScript với các cấu hình tối ưu, giúp phát triển mã nguồn nhanh chóng và hiệu quả.

0.3.2. *Server-Side Rendering (SSR) và Lợi ích SEO*

Server-Side Rendering (SSR) là cơ chế kết xuất giao diện ngay trên máy chủ trước khi gửi nội dung HTML hoàn chỉnh đến trình duyệt người dùng. Việc áp dụng SSR mang lại nhiều lợi ích:

- Tăng tốc độ hiển thị nội dung ban đầu của trang Web.
- Hỗ trợ các công cụ tìm kiếm dễ dàng thu thập và lập chỉ mục dữ liệu.
- Cải thiện SEO trên các công cụ tìm kiếm và tối ưu trải nghiệm người dùng.

tai_nguyen/so-do-mo-ta-luong-xu-ly-cua-ssr-server-side-rendering.png

Hình 3. Sơ đồ mô tả luồng xử lý của SSR (Server-Side Rendering)

Luồng hoạt động SSR được chia thành các giai đoạn tuần tự:

1. **Khởi tạo yêu cầu (Request):** Trình duyệt gửi yêu cầu HTTP tới máy chủ để truy cập vào một tài nguyên cụ thể.

2. **Thu thập dữ liệu và Kết xuất tại máy chủ (Fetch & Render):** Máy chủ truy vấn cơ sở dữ liệu hoặc API nội bộ, sau đó kết xuất các React Component thành tệp HTML hoàn chỉnh.
3. **Hiển thị nội dung tức thời (First Paint):** Trình duyệt nhận HTML đã có sẵn nội dung và hiển thị ngay lập tức, mang lại lợi thế SEO tuyệt đối.
4. **Thủy hợp (Hydration):** React khởi tạo cây Virtual DOM và đính kèm các Event Listeners vào các phần tử HTML tĩnh hiện có.
5. **Trạng thái tương tác hoàn chỉnh (Fully Interactive):** Ứng dụng chuyển đổi thành Single Page Application (SPA), người dùng tương tác mượt mà mà không cần tải lại trang.

Bên cạnh SSR, Next.js còn hỗ trợ Client-Side Rendering (CSR): dữ liệu được xử lý và hiển thị trực tiếp trên trình duyệt, phù hợp với các ứng dụng yêu cầu khả năng tương tác cao hoặc cập nhật dữ liệu realtime.

0.4. Giao thức giao tiếp RESTful API

REST (Representational State Transfer) là một kiểu kiến trúc phần mềm tiêu chuẩn chi phối cách thức các dịch vụ web giao tiếp với nhau qua Internet. Hệ thống tuân thủ REST (RESTful API) sử dụng giao thức HTTP/HTTPS làm phương tiện truyền tải thông điệp, thường định dạng dữ liệu dưới chuẩn JSON (JavaScript Object Notation).

Một hệ thống RESTful hoạt động dựa trên nguyên tắc phi trạng thái (Stateless), nghĩa là máy chủ không lưu trữ thông tin về phiên làm việc của máy khách giữa các yêu cầu. Các phương thức HTTP cơ bản như GET (truy xuất), POST (tạo mới/gửi lệnh), PUT (cập nhật) và DELETE (xóa bỏ) được ánh xạ trực tiếp vào các hành động tương tác với tài nguyên hệ thống, tạo ra một chuẩn mực giao tiếp thống nhất giữa nền tảng web, cổng thanh toán trung gian và thiết bị vi điều khiển IoT.

0.5. Công nghệ IoT và Vi điều khiển ESP32

ESP32 là một thiết bị phần cứng hiệu năng cao, tích hợp sẵn Wi-Fi và Bluetooth, rất phù hợp cho các hệ thống Internet of Things (IoT). Dòng chip này nổi bật với tốc độ tính toán cao cùng các phương thức giao tiếp mạng đa dạng.



tai_nguyen/so-do-chi-tiet-cac-chan-chuc-nang-gpio-tren-bo-mach-esp32-ch

Hình 4. Sơ đồ chi tiết các chân chức năng GPIO trên bo mạch ESP32 chuẩn

Một số ứng dụng thực tế phổ biến của ESP32:

- **Nhà thông minh (Smart Home):** Điều khiển đèn, điều hòa, rèm cửa tự động và các thiết bị khác thông qua Wi-Fi hoặc Bluetooth.
- **Nông nghiệp thông minh (Smart Agriculture):** Theo dõi độ ẩm đất, nhiệt độ, ánh sáng và tự động hóa hệ thống tưới tiêu.
- **Hệ thống an ninh:** Tích hợp với camera (như ESP32-CAM), cảm biến chuyển động và hệ thống báo động.

Vai trò của ESP32 trong hệ thống ACW-SRS:

- Đóng vai trò là thiết bị thực thi (Actuator) tại các trạm rửa xe vật lý.
- Tiếp nhận lệnh điều khiển từ Server Next.js thông qua giao thức HTTP/HTTPS.
- Điều khiển các module Relay và liên tục gửi báo cáo trạng thái về trung tâm.

0.6. Hệ quản trị Cơ sở dữ liệu (MySQL & Prisma ORM)

0.6.1. Hệ quản trị cơ sở dữ liệu MySQL

Để xử lý và lưu trữ luồng thông tin có tính ràng buộc phức tạp, hệ thống sử dụng MySQL – một trong những hệ quản trị cơ sở dữ liệu quan hệ (Relational Database Management System) mã nguồn mở phổ biến nhất hiện nay. MySQL cho phép quản lý dữ liệu thông qua ngôn ngữ truy vấn SQL và tổ chức dữ liệu dưới dạng bảng có liên kết với nhau.

Trong dự án, MySQL được sử dụng để:

- Lưu trữ thông tin người dùng và dữ liệu hệ thống.
- Quản lý các mối quan hệ giữa các bảng dữ liệu.
- Hỗ trợ truy vấn, thêm, sửa và xóa dữ liệu một cách hiệu quả.

0.6.2. Prisma ORM và lợi ích trong phát triển hệ thống

Prisma là một bộ công cụ cơ sở dữ liệu hiện đại, giúp lập trình viên thao tác với dữ liệu hiệu quả hơn thông qua:

- **Tự động tạo mã:** Dựa trên sơ đồ dữ liệu (Schema), Prisma tự động tạo ra thư viện truy vấn mạnh mẽ, hỗ trợ đầy đủ TypeScript.
- **Truy vấn an toàn:** Loại bỏ các lỗi cú pháp SQL và đảm bảo dữ liệu truy vấn luôn đúng kiểu, giúp giảm thiểu lỗi thời gian chạy (Runtime errors).
- **Tính năng Migration:** Hỗ trợ đồng bộ cấu trúc dữ liệu giữa cơ sở dữ liệu và mã nguồn thực tế một cách có kiểm soát.

Ví dụ về Prisma Schema (định nghĩa dữ liệu):

```
model Station {
  uuid          Int          @id @default(autoincrement())
  station_name  String
  status        String       @default("OFFLINE")
  location      String?
  created_at    DateTime     @default(now())
}
```

Ví dụ về Prisma Client (truy vấn dữ liệu):

```
// Lay danh sach tat ca cac tram dang hoạt động
const activeStations = await prisma.station.findMany({
  where: { status: "ONLINE" }
});
```

Việc sử dụng Prisma giúp loại bỏ các câu lệnh SQL thô (raw SQL) phức tạp, thay vào đó là các hàm gọi trực quan, giúp tăng tốc độ phát triển và giảm thiểu sai sót về logic.

dữ liệu.

0.7. Công nghệ Thanh toán và Giải pháp SePay tự động

0.7.1. QR Dynamic và Chuẩn VietQR (NAPAS 247)

Trong hệ thống ACW-SRS, thanh toán phải diễn ra chính xác và tự động hoàn toàn. Dự án sử dụng tiêu chuẩn VietQR để tạo ra các mã QR động cho mỗi trạm. Khách hàng chỉ cần quét mã, mọi thông tin về số tài khoản, số tiền và nội dung chuyển khoản sẽ được điền tự động, loại bỏ hoàn toàn nguy cơ nhập sai thông tin.

0.7.2. Giải pháp SePay và Cơ chế Webhook tự động hóa

SePay là giải pháp trung gian kết nối giữa ngân hàng và hệ thống phần mềm. Cơ chế Webhook cho phép SePay gửi thông tin giao dịch tự động cho Server Next.js ngay khi tài khoản ngân hàng nhận được tiền. Ví dụ payload SePay gửi về server:

```
{
  "id": 909,
  "gateway": "Vietabank",
  "transactionDate": "2022-06-21 12:07:50",
  "accountNumber": "0999888787",
  "content": "ACW-SRS-TRAM01",
  "transferType": "in",
  "transferAmount": 15000,
  "referenceCode": "ABC.92754"
}
```



tai_nguyen/so-do-luong-hoat-dong-cua-giai-phap-thanh-toan-trung-gian-sepa

Hình 5. Sơ đồ luồng hoạt động của giải pháp thanh toán trung gian SePay

Sơ đồ trên trình bày kiến trúc tích hợp và luồng luân chuyển dữ liệu tự động của dịch vụ thanh toán SePay, bao gồm ba phân lớp xử lý:

1. **Lớp tiếp nhận sự kiện:** Mạng lưới liên kết với hơn 15 ngân hàng lớn. Bất cứ khi nào tài khoản thụ hưởng phát sinh giao dịch ghi có, hệ thống ngân hàng sẽ tạo ra tín hiệu biến động số dư.
2. **Lớp trung gian phân tích (Lõi SePay):** Toàn bộ dữ liệu thô từ ngân hàng được tập trung về máy chủ SePay, tự động trích xuất, phân tích và chuẩn hóa các trường thông tin như số tiền, nội dung chuyển khoản và mã tham chiếu.
3. **Lớp phân phối và tích hợp:** SePay sử dụng Webhooks để đẩy (Push) thông tin đến Server Next.js, kích hoạt máy rửa xe tự động.